

4장

수의 표현

3장까지는 명제와 집합을 다루었습니다. 이 장은 이산수학의 또 다른 재료인 수를 다룹니다. 같은 수를 어떤 방식으로 적을 수 있고, 수 하나에 쓸 비트 수가 정해진 컴퓨터는 음수와 뺄셈을 어떻게 처리할까요?

1절은 자연수에서 복소수까지 수의 종류와, 수의 연산이 따르는 성질을 정리합니다. 2절은 같은 수를 10진수, 2진수, 8진수, 16진수로 적는 수 체계와 그 사이의 변환을 다룹니다. 3절은 정해진 비트 안에서 음수를 나타내는 부호화-절대치 표현과 보수 표현을, 4절은 보수를 써서 뺄셈을 덧셈으로 계산하는 방법을 다룹니다. 읽는 내내 한 가지 식을 눈여겨보세요. 1절에서 처음 나오는 $n = a_k b^k + \dots + a_1 b^1 + a_0 b^0$, 곧 각 자리의 숫자에 그 자리의 값을 곱해 더하는 식입니다. 2절의 변환은 이 식을 다른 기수로 다시 쓰는 일이고, 3절과 4절의 보수는 n 비트 바로 위의 자리값 2^n 을 기준으로 삼습니다.

1. 수와 그 성질

1-1) 수의 종류

3장 그림 3-1에서 수의 집합 N, Z, Q, I, R, C 의 포함 관계를 보았습니다. 이 소절은 그 수들을 하나씩 정의하고, 각 수를 자리마다 숫자로 적는 방법을 함께 봅니다.

자연수(natural number)

1, 2, 3, ...처럼 사물을 셀 때 쓰는 양의 정수를 자연수라 하고, 그 집합을 N 으로 씁니다. 자연수 n 은 기수(base) b 를 정하면($b \in N, b > 1$) 다음과 같이 나타낼 수 있고, 이렇게 적은 수를 n_b 로 씁니다.

$$n = a_k b^k + a_{k-1} b^{k-1} + \dots + a_1 b^1 + a_0 b^0$$

여기서 각 a_i 는 $0 \leq a_i < b$ 인 정수이고, 맨 앞 숫자 a_k 는 0이 아닙니다.

a_i 는 오른쪽에서 i 번째 자리의 숫자이고(맨 오른쪽을 0번째로 셉니다), b^i 는 그 자리의 값입니다. 10진수로 말하면 $b^0 = 1$ 은 일의 자리, $b^1 = 10$ 은 십의 자리, $b^2 = 100$ 은 백의 자리입니다. 맨 윗자리의 지수가 k 이므로 숫자는 a_k 부터 a_0 까지 $k + 1$ 개입니다. 589는 $k = 2$ 이고 숫자가 셋입니다. 수를 이렇게 각 자리의 숫자와 기수의 거듭제곱으로 풀어 쓰는 것을 '기수와 자리수로 표현한다'고 합니다.

조건 $0 \leq a_i < b$ 에는 이유가 있습니다. 10진수의 한 자리에 쓸 수 있는 숫자는 0부터 9까지이고 10은 쓸 수 없습니다. 10이 되면 윗자리로 1을 올리기 때문입니다. 마찬가지로 기수가 b 이면 한 자리의 숫자는 b 보다 작아야 합니다. 기수가 1보다 커야 하는 까닭도 여기서 나옵니다. $b = 1$ 이면 쓸 수 있는 숫자가 0 하나뿐이어서 어떤 양의 정수도 적을 수 없습니다. 맨 앞 숫자를 0이 아니게 정한 것은 0589와 589처럼 같은 수가 두 가지로 적히지 않게 하기 위해서입니다.

기수라는 말은 3장에서 집합의 원소의 개수(cardinality)를 뜻했습니다. 여기서는 자리값의 밑이 되는 수(base)를 뜻합니다. 같은 낱말이 다른 뜻으로 쓰이므로, 집합에 붙은 말인지 수 체계에 붙은 말인지 보고 구별하세요. 수의 오른쪽 아래에 작게 붙인 10, 2 같은 숫자가 이 기수입니다. 589_{10} 은 '10진수 589'라고 읽습니다.

예제 4-1

589_{10} 을 기수와 자리수로 표현하시오.

풀이

기수는 10입니다. 맨 오른쪽의 9가 일의 자리 숫자 a_0 이고, 그 왼쪽의 8이 a_1 , 5가 a_2 입니다. 자리값은 차례로 10^0 , 10^1 , 10^2 입니다.

$$589_{10} = 5 \times 10^2 + 8 \times 10^1 + 9 \times 10^0$$

계산하면 $500 + 80 + 9 = 589$ 로 돌아옵니다.

정수(integer)

양의 정수, 0, 음의 정수로 이루어진 수 체계를 정수라 하고, 그 집합을 Z 로 씁니다.

양의 정수가 곧 자연수이므로, 정수는 자연수에 0과 $-1, -2, -3, \dots$ 을 더한 것입니다. 음의 정수 $-n$ 은 자연수 n 의 자리 표현 앞에 부호를 붙여 적습니다.

유리수(rational number)

$a, b \in Z$ 이고 $a \neq 0$ 일 때 $\frac{b}{a}$ 로 나타낼 수 있는 수를 유리수라 하고, 그 집합을 Q 로 씁니다.

분모 a 가 0이 아니어야 하는 것은 0으로 나누는 일이 정의되지 않기 때문입니다. 정수 n 은 $\frac{n}{1}$ 로 쓸 수 있으므로 모든 정수는 유리수입니다.

같은 유리수를 여러 분수로 쓸 수 있습니다. $\frac{6}{8}$ 과 $\frac{3}{4}$ 은 같은 수입니다. 분모와 분자를 공약수로 끝까지 나누어, 분모와 분자 사이에 1 말고는 공약수가 없게 만든 꼴을 하한항(lowest terms)이라고 합니다. 고등학교에서 기약분수라고 부르던 것입니다. 3장에서 말한 정수의 서로소로 말하면, 하한항은 분자와 분모가 서로소인 분수입니다. $\frac{3}{4}$ 이 $\frac{6}{8}$ 의 하한항입니다.

유리수를 소수로 나타내면 소수점 아래 부분, 곧 소수부의 숫자들이 유한하게 끝나거나 일정하게 반복됩니다. $\frac{1}{2} = 0.5$ 는 끝나고, $\frac{1}{3} = 0.333\cdots$ 은 3이, $\frac{1}{7} = 0.142857142857\cdots$ 은 142857이 반복됩니다. 반복이 생기는 까닭은 나눗셈에 있습니다. 분모 a 를 양수로 놓고(부호는 분자로 옮기면 됩니다) b 를 a 로 나누는 긴 나눗셈을 하면, 단계마다 나오는 나머지는 0부터 $a - 1$ 까지 가운데 하나입니다. 나머지가 0이 되면 나눗셈이 끝나고, 0이 되지 않으면 나머지의 종류가 유한하므로 언젠가 앞에서 나온 나머지가 다시 나옵니다. 같은 나머지에서 다시 나누면 그다음 숫자들도 똑같이 되풀이됩니다. 소수부를 실수부라고 부르지 않도록 주의하세요. 실수부는 아래 복소수에서 따로 쓰는 말입니다.

무리수(irrational number)

$a, b \in \mathbb{Z}$ 이고 $a \neq 0$ 일 때 $\frac{b}{a}$ 로 나타낼 수 없는 실수를 무리수라 하고, 그 집합을 I 로 씁니다.

$\sqrt{2} = 1.41421356\cdots$ 이 대표적인 무리수입니다. 무리수의 소수부는 끝나지 않고, 어디서부터도 일정하게 반복되지 않습니다. 반복되는 소수는 분수로 나타낼 수 있어서 유리수이기 때문입니다. 숫자가 아무렇게나 나온다는 뜻은 아닙니다. $\sqrt{2}$ 의 숫자는 몇째 자리까지든 정확히 계산해 낼 수 있습니다. 다만 같은 덩어리가 되풀이되는 일이 없을 뿐입니다.

실수(real number)

유리수와 무리수를 모두 포함하는 수 체계를 실수라 하고, 그 집합을 R 로 씁니다. $R = \mathbb{Q} \cup I$ 입니다.

자연수와 정수는 유리수에 들어 있으므로($\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q}$), 자연수, 정수, 유리수, 무리수를 모두 합한 $\mathbb{N} \cup \mathbb{Z} \cup \mathbb{Q} \cup I$ 도 $\mathbb{Q} \cup I$ 와 같은 집합입니다. 실수는 자연수, 정수, 유리수, 무리수를 모두 포함합니다.

실수는 소수점 아래로 자리가 이어지므로 자리 표현도 소수점 아래로 늘어납니다. 0 이상의 실수 r 과 기수 $b(b \in \mathbb{N}, b > 1)$, $0 \leq a_i < b$ 인 숫자 a_i 에 대해 다음과 같습니다.

$$r = a_k b^k + \cdots + a_1 b^1 + a_0 b^0 + a_{-1} b^{-1} + a_{-2} b^{-2} + \cdots$$

소수점 왼쪽은 지수가 0, 1, 2, ...로 커지고, 오른쪽은 -1, -2, ...로 작아집니다. $b^{-1} = \frac{1}{b}$ 이므로 소수점 첫째 자리의 자리값은 1을 기수로 한 번 나눈 값, 둘째 자리는 두 번 나눈 값입니다. 10진수로는 $10^{-1} = 0.1$, $10^{-2} = 0.01$ 입니다. 소수부가 끝나는 유리수는 이 합이 어느 자리에서 끝나고, 소수부가 반복되는 유리수와 무리수는 끝없이 이어집니다. 음의 실수는 이 표현 앞에 부호를 붙입니다.

예제 4-5

345.734₁₀을 기수와 자리수로 표현하시오.

풀이

소수점을 기준으로 양쪽으로 지수가 하나씩 멀어진다고 보면 됩니다. 소수점 왼쪽은 오른쪽부터 $10^0, 10^1, 10^2$ 의 자리이고, 소수점 오른쪽은 왼쪽부터 $10^{-1}, 10^{-2}, 10^{-3}$ 의 자리입니다.

$$345.734_{10} = 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0 \\ + 7 \times 10^{-1} + 3 \times 10^{-2} + 4 \times 10^{-3}$$

복소수(complex number)

제공하여 -1 이 되는 수 i 를 포함하는 수 체계를 복소수라 하고, 그 집합을 C 로 씁니다. 복소수 c 는 실수 a , b 로 $c = a + bi$ 처럼 나타내며, a 를 c 의 실수부(real part), b 를 c 의 허수부(imaginary part)라고 합니다.

i 는 $i^2 = -1$ 을 만족하는 수로, 허수 단위(imaginary unit)라고 부릅니다. 실수는 제공하면 0 이상이므로 $x^2 = -1$ 인 실수 x 는 없습니다. 복소수는 이 방정식에도 답이 있도록 수를 넓힌 것입니다. 허수부는 bi 가 아니라 i 앞의 실수 b 라는 점에 주의하세요. $3 + 6i$ 의 허수부는 6입니다. $b = 0$ 이면 $a + 0i = a$ 이므로 실수도 복소수이고, 3장에서 허수라고 부른 것은 $b \neq 0$ 인 복소수입니다.

복소수의 덧셈은 실수부는 실수부끼리, 허수부는 허수부끼리 더합니다. 곱셈은 다항식처럼 각 항을 모두 곱해 펼친 뒤 i^2 을 -1 로 바꿉니다. 아래에서 a, b, c, d 는 실수입니다.

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

$$(a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i$$

곱셈의 실수부에 $-bd$ 가 생기는 것은 bdi^2 이 $i^2 = -1$ 때문에 $-bd$ 라는 실수가 되어 실수부로 옮겨가기 때문입니다.

예제 4-6

복소수 $3 + 6i$ 와 $12i$ 에 대해 합과 곱은?

풀이

$12i$ 는 실수부가 0이고 허수부가 12인 복소수 $0 + 12i$ 입니다. 합은 실수부가 $3 + 0 = 3$, 허수부가 $6 + 12 = 18$ 이므로 $3 + 18i$ 입니다. 곱은 각 항을 곱해 펼친 뒤 $i^2 = -1$ 을 씁니다.

$$(3 + 6i) \times 12i = 36i + 72i^2 = 36i - 72 = -72 + 36i$$

$\therefore$ 합은 $3 + 18i$, 곱은 $-72 + 36i$ 입니다.

지금까지의 수를 포함 관계로 그리면 그림 4-1과 같습니다. 가장 안쪽의 자연수에 0과 음의 정수를 더하면 정수가 되고, 정수에 분수로만 나타나는 수를 더하면 유리수가 됩니다. 유리수와 무리수를 합하면 실수이고, 실수와 허수를 합하면 복소수입니다. 3장 그림 3-1과 같은 관계를 원으로 그린 것입니다.

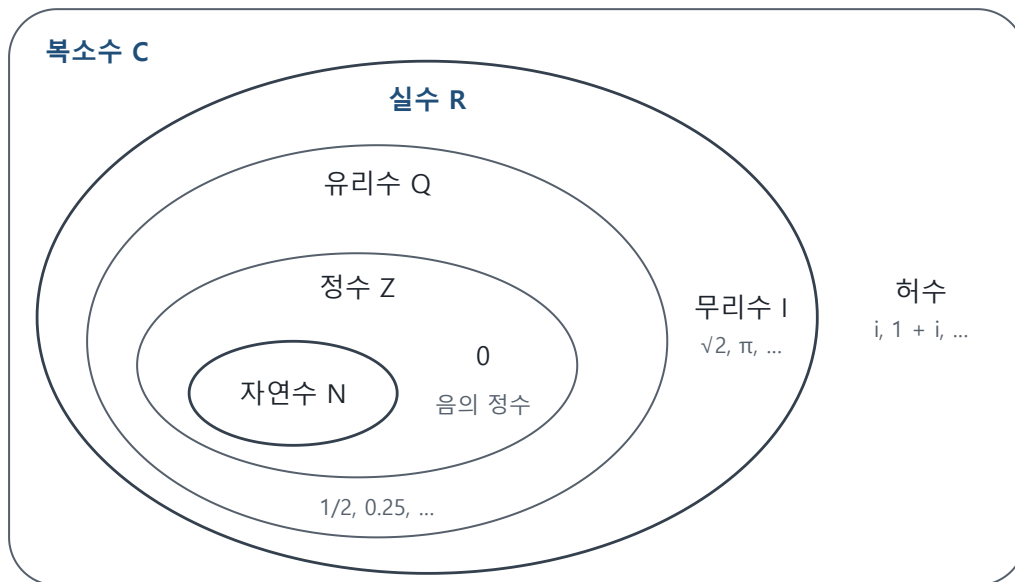


그림 4-1 수의 종류와 포함 관계

1-2) 수의 연산

수의 닫힘 성질

2장에서 어떤 수의 모임 안에서 계산한 결과가 늘 그 모임 안에 남으면 그 모임이 그 연산에 닫혀 있다고 했습니다. 정수는 덧셈, 뺄셈, 곱셈에는 닫혀 있지만 나눗셈에는 닫혀 있지 않았습니다. 이 성질을 닫힘 성질(closure)이라고 합니다. 정확히 말하면, 어떤 수 체계에 속한 임의의 두 원소를 연산한 결과가 언제나 그 수 체계에 속할 때 그 수 체계는 그 연산에 닫혀 있다고 합니다. '언제나'가 핵심입니다. 한 번이라도 밖으로 나가는 경우가 있으면 닫혀 있지 않습니다. 표 4-1은 수 체계마다 사칙연산에 닫혀 있는지를 정리한 것입니다.

수 체계	덧셈	뺄셈	곱셈	나눗셈
자연수(N)	○	×	○	×
정수(Z)	○	○	○	×
유리수(Q)	○	○	○	○
무리수(I)	×	×	×	×
실수(R)	○	○	○	○
복소수(C)	○	○	○	○

표 4-1 수의 닫힘 성질. ○는 닫혀 있음, ×는 닫혀 있지 않음이고, 나눗셈은 0으로 나누는 경우를 뺍니다.

자연수는 더하거나 곱해도 자연수이지만, 빼면 $3 - 5 = -2$ 처럼, 나누면 $3 \div 2 = \frac{3}{2}$ 처럼 자연수 밖으로 나갈 수 있습니다. 정수는 뺄셈까지 닫혀 있지만, $1 \div 2$ 가 정수가 아니므로 나눗셈에는 닫혀 있지 않습니다. 유리수는 0으로 나누는 경우만 빼면 네 연산에 모두 닫혀 있습니다. 두 분수를 더하거나 빼거나 곱하거나 나누어도, 결과는 다시 정수를 분자와 분모로 하는 분수가 되기 때문입니다. 실수와 복소수도 네 연산에 모두 닫혀 있습니다.

눈여겨볼 것은無理수입니다.無理수끼리 계산하면 유리수가 나올 수 있어서 네 연산 어디에도 닫혀 있지 않습니다. $\sqrt{2} + (-\sqrt{2}) = 0$, $\sqrt{2} - \sqrt{2} = 0$, $\sqrt{2} \times \sqrt{2} = 2$, $\sqrt{2} \div \sqrt{2} = 1$ 처럼 결과가 모두 유리수입니다. 닫혀 있지 않음을 보이는 데는 이런 예 하나면 충분합니다. 2장의 반례증명법과 같은 생각입니다.

수의 합과 곱에 대한 특징

수의 덧셈과 곱셈은 다음 법칙을 따릅니다. 1장에서 명제의 연산이, 3장에서 집합의 연산이 따르던 교환법칙, 결합법칙, 분배법칙과 이름도 꼴도 같습니다. xy 는 $x \times y$ 를 줄여 쓴 것입니다.

- 교환법칙: $x + y = y + x$, $xy = yx$
- 결합법칙: $(x + y) + z = x + (y + z)$, $(xy)z = x(yz)$
- 분배법칙: $x(y + z) = xy + xz$

분배법칙은 곱셈이 괄호 안의 덧셈 위로 나누어 들어간다는 뜻입니다. 거꾸로 덧셈이 곱셈 위로 나누어 들어가는 $x + yz = (x + y)(x + z)$ 는 수에서 성립하지 않습니다. $x = y = z = 1$ 을 넣어 보면 양변이 다릅니다.

$$1 + 1 \times 1 = 2, \quad (1 + 1)(1 + 1) = 4$$

명제와 집합에서는 두 방향의 분배법칙이 모두 성립했으니, 수의 법칙과 명제나 집합의 법칙을 서로 옮길 때 이 차이를 기억해 두세요.

항등원(identity element)

어떤 수 a 에 연산한 결과가 언제나 a 자신이 되게 하는 수를 그 연산의 항등원이라고 합니다. 합에 대한 항등원은 0이고($a + 0 = a$), 곱에 대한 항등원은 1입니다($a \times 1 = a$).

역(inverse), 역원

어떤 수 a 와 연산한 결과가 그 연산의 항등원이 되게 하는 수를 a 의 역이라고 합니다. 합에 대한 역은 $-a$ 이고($a + (-a) = 0$), 곱에 대한 역은 $\frac{1}{a}$ 입니다($a \times \frac{1}{a} = 1$, $a \neq 0$).

역은 항등원을 거쳐 정의됩니다. 먼저 아무것도 바꾸지 않는 수인 항등원을 정하고, 그다음 연산해서 그 항등원으로 되돌려 놓는 수를 역이라 부르는 것입니다. 0은 무엇을 곱해도 0이므로 곱해서 1이 되는 수가 없습니다. 그래서 곱에 대한 역은 0이 아닌 수에만 있습니다. 또 정수의 범위에서는 곱에

대한 역이 정수인 수가 1과 -1뿐입니다. 2의 역 $\frac{1}{2}$ 은 정수가 아닙니다. 나눗셈은 역을 곱하는 일이므로($x \div a = x \times \frac{1}{a}$), 이것이 곧 표 4-1에서 정수가 나눗셈에 닫혀 있지 않던 까닭입니다.

예제 4-7

9의 합과 곱에 대한 역을 구하시오.

풀이

합에 대한 역은 9에 더해서 합에 대한 항등원 0이 되는 수입니다. $9 + a = 0$ 에서 $a = -9$ 입니다.

곱에 대한 역은 9에 곱해서 곱에 대한 항등원 1이 되는 수입니다. $9 \times a = 1$ 에서 $a = \frac{1}{9}$ 입니다.

$\therefore$ 합에 대한 역은 -9, 곱에 대한 역은 $\frac{1}{9}$ 입니다.

합의 표시와 곱의 표시

일정한 규칙으로 나열된 값을 모두 더하거나 곱할 때는 기호를 써서 짧게 씁니다.

$$\sum_{i=1}^n x_i = x_1 + x_2 + x_3 + \cdots + x_i + \cdots + x_{n-1} + x_n$$

$$\prod_{i=1}^n x_i = x_1 \times x_2 \times x_3 \times \cdots \times x_i \times \cdots \times x_{n-1} \times x_n$$

Σ 는 '시그마', Π 는 '파이'라고 읽습니다. 각각 합(sum)과 곱(product)의 첫 글자에 해당하는 그리스 문자의 대문자이며, Π 는 원주율 π 와 다른 기호입니다. 아래의 $i = 1$ 은 i 를 1부터 시작한다는 뜻이고, 위의 n 은 n 에서 끝난다는 뜻입니다. i 처럼 1, 2, 3, ...으로 바뀌며 몇 번째 값인지 가리키는 문자를 합이나 곱의 색인(index)이라고 합니다. 예를 들어 $x_i = i$ 이면 다음과 같습니다.

$$\sum_{i=1}^4 i = 1 + 2 + 3 + 4 = 10, \quad \prod_{i=1}^4 i = 1 \times 2 \times 3 \times 4 = 24$$

1-1)의 자리 표현도 이 기호로 짧게 쓸 수 있습니다. $n = \sum_{i=0}^k a_i b^i$ 이고, 이번에는 색인이 0부터 시작합니다.

나누기 연산

나누기 연산 $d \mid n$

정수 n 과 0이 아닌 정수 d 에 대해 $n = dq$ 를 만족하는 정수 q 가 있을 때, 곧 n 이 d 로 나누어떨어질 때 $d \mid n$ 으로 씁니다. 그렇지 않을 때는 $d \nmid n$ 으로 씁니다. 이때 q 를 몫(quotient), d 를 n 의 약수(divisor) 또는 인수(factor), n 을 d 의 배수(multiple)라고 합니다.

$d \mid n$ 은 'd는 n을 나눈다'라고 읽고, $d \nmid n$ 은 'd는 n을 나누지 못한다'라고 읽습니다. 나누는 수를 제수, 나누어지는 수를 피제수라고도 하므로 d 가 제수, n 이 피제수입니다. 예를 들어 $12 = 3 \times 4$ 이므로

$3 \mid 12$ 이고 몫은 4입니다. 12는 5로 나누어떨어지지 않으므로 $5 \nmid 12$ 입니다.

이 기호에서 가장 헷갈리기 쉬운 것은 $d \mid n$ 이 수가 아니라는 점입니다. $d \mid n$ 은 참이나 거짓이 정해지는 명제입니다. $3 \mid 12$ 는 참인 명제이고, 분수 $3/12$ 는 0.25라는 수입니다. 나누는 수의 자리도 분수와 반대입니다. 분수 $\frac{a}{b}$ 에서는 나누는 수 d 가 아래에 있지만 $d \mid n$ 에서는 세로선 왼쪽에 옵니다. $d \neq 0$ 을 요구하는 것은 0으로 나누는 일이 정의되지 않기 때문입니다. 반대로 $n = 0$ 은 괜찮습니다. $0 = d \times 0$ 이므로 0이 아닌 모든 d 에 대해 $d \mid 0$ 입니다.

2장에서 짝수를 어떤 정수 k 에 대해 $n = 2k$ 인 수로 정의했습니다. 이 장의 기호로는 $2 \mid n$ 입니다. 정의가 ' $n = dq$ 인 정수 q 가 있다'는 꼴이어서, 나누기 연산에 관한 성질은 2장의 직접증명법처럼 정의를 식으로 옮겨 증명합니다.

나누기 연산의 규칙

a, b, c, d, m, n 이 정수일 때 다음 규칙이 성립합니다. 증명은 모두 같은 방법입니다. 나눈다는 조건을 몫이 되는 정수가 있다는 식으로 바꾼 뒤, 결론의 수가 '나누는 수 $\times$ 정수' 꼴이 됨을 보입니다.

(1) $d \mid m$ 이고 $d \mid n$ 이면 $d \mid (m + n)$ 가정 $d \mid m$ 에서 어떤 정수 k 에 대해 $m = dk$ 이고, 가정 $d \mid n$ 에서 어떤 정수 l 에 대해 $n = dl$ 입니다. 더하면 $m + n = dk + dl = d(k + l)$ 이고, 정수는 덧셈에 닫혀 있으므로 $k + l$ 은 정수입니다. 따라서 $d \mid (m + n)$ 입니다.

(2) $d \mid m$ 이고 $d \mid n$ 이면 $d \mid (m - n)$ 같은 방법으로 $m - n = dk - dl = d(k - l)$ 이고 $k - l$ 은 정수입니다.

(3) $d \mid m$ 이면 $d \mid mn$ 가정에서 어떤 정수 k 에 대해 $m = dk$ 이므로 $mn = dkn = d(kn)$ 이고 kn 은 정수입니다. 어떤 수의 배수에 무엇을 곱해도 여전히 그 수의 배수라는 뜻입니다.

(4) $a \mid b$ 이고 $b \mid c$ 이면 $a \mid c$ 가정에서 어떤 정수 k, l 에 대해 $b = ak, c = bl$ 이므로 $c = (ak)l = a(kl)$ 이고 kl 은 정수입니다. 3이 6을 나누고 6이 24를 나누면 3이 24를 나누는 식입니다. 3장 포함관계 정리 (4)의 ' $A \subseteq B$ 이고 $B \subseteq C$ 이면 $A \subseteq C$ '와 같은 꼴의 규칙입니다.

(1)과 (2)의 증명에서 두 몫에 서로 다른 문자 k, l 을 쓴 데 주의하세요. m 과 n 이 둘 다 d 의 배수라도 몫까지 같다는 보장은 없습니다. 두 몫을 같은 k 로 쓰면 $m = n$ 인 경우만 증명한 셈이 됩니다.

나머지 연산

나머지 연산 $n \bmod d$

정수 n 을 양의 정수 d 로 나눈 몫이 q 이고 나머지가 r 일 때, 곧 $n = dq + r$ 이고 $0 \leq r < d$ 일 때, 나머지 r 을 구하는 연산을 $n \bmod d = r$ 로 씁니다. r 을 나머지(remainder)라고 합니다.

$\bmod$ 는 '모듈로'(modulo)라고 읽습니다. $17 \bmod 5$ 는 '17 모듈로 5'라고 읽고, $17 = 5 \times 3 + 2$ 이므로 값은 2입니다. 앞의 $d \mid n$ 이 참과 거짓을 정하는 명제였던 것과 달리, $n \bmod d$ 는 수 하나를 결과로

내는 연산입니다. 여기서 d 는 나누는 수일 뿐 n 의 약수가 아닙니다. 17을 나누는 5는 17의 약수가 아닙니다.

나머지의 조건 $0 \leq r < d$ 가 핵심입니다. $17 = 5 \times 2 + 7$ 처럼 쓸 수도 있지만 7은 5보다 크므로 한 번 더 나눌 수 있습니다. 나머지가 d 보다 작아질 때까지 나누어야 몫과 나머지가 하나로 정해집니다. 이 조건 덕분에 음수도 같은 방법으로 처리됩니다. $-7 \bmod 3$ 은 $-7 = 3 \times (-3) + 2$ 이므로 2입니다. $-7 = 3 \times (-2) - 1$ 로 쓰면 나머지가 -1 이 되어 조건 $0 \leq r$ 을 어깁니다.

나머지가 0이면 나누어떨어진다는 뜻이므로 다음이 성립합니다.

$$n \bmod d = 0 \Leftrightarrow d \mid n$$

$n \bmod d = 0$ 이면 $n = dq$ 이므로 $d \mid n$ 이고, 거꾸로 $d \mid n$ 이면 $n = dq + 0$ 이고 0은 $0 \leq r < d$ 를 만족하므로 나머지가 0입니다. 2장의 짝수와 홀수로 말하면 짝수는 $n \bmod 2 = 0$ 인 수이고 홀수는 $n \bmod 2 = 1$ 인 수입니다. 2절에서 10진수를 2진수로 바꿀 때는 2로 나눈 이 나머지가 그대로 2진수의 숫자가 됩니다.

2. 수 체계

1절에서 수를 기수 b 의 거듭제곱으로 풀어 적는 방법을 보았습니다. 기수를 무엇으로 정하느냐에 따라 같은 수가 다른 숫자열로 적힙니다. 기수를 정해 수를 적는 방식을 수 체계라고 하며, 기수가 10이면 10진수, 2이면 2진수입니다. 일상에서는 10진수를 쓰고, 컴퓨터는 0과 1 두 상태로 정보를 다루므로 2진수를 씁니다. 2진수는 자릿수가 금방 길어져 사람이 읽고 주고받기 번거롭기 때문에, 2진수를 짧게 묶어 적는 16진수와 8진수를 함께 씁니다. 이 가운데 10진수, 2진수, 16진수가 특히 자주 쓰입니다.

10진수(decimal number)

기수를 10으로 하는 수 체계로, 0에서 9 사이의 숫자를 이용해 수를 나타냅니다.

$$\begin{aligned} n_{10} &= a_k a_{k-1} \cdots a_1 a_0 . a_{-1} a_{-2} \cdots \\ &= a_k 10^k + \cdots + a_1 10^1 + a_0 10^0 + a_{-1} 10^{-1} + a_{-2} 10^{-2} + \cdots \end{aligned}$$

첫 줄은 숫자를 늘어놓은 모양이고, 둘째 줄은 그 숫자열의 뜻입니다. 첫 줄의 $a_k a_{k-1}$ 은 곱이 아니라 숫자를 나란히 적은 것이라는 점에 주의하세요. 소수점은 a_0 과 a_{-1} 사이에 찍힙니다. 정수이면 소수점 아래가 없고, 소수부가 유한한 수이면 어느 자리 a_{-l} 에서 끝납니다.

2진수(binary number)

기수를 2로 하는 수 체계로, 0과 1만 이용해 수를 나타냅니다.

$$\begin{aligned}n_2 &= a_k a_{k-1} \cdots a_1 a_0 . a_{-1} a_{-2} \cdots \\&= a_k 2^k + \cdots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + a_{-2} 2^{-2} + \cdots\end{aligned}$$

10진수와 똑같은 식에서 기수만 2로 바뀌었습니다. 소수점 위로는 자리값이 1, 2, 4, 8, ...로 두 배씩 커지고, 아래로는 $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots$ 로 반씩 작아집니다. 2진수의 한 자리가 비트입니다. 2진수 101은 '백일'이 아니라 '일 영 일'처럼 자리마다 읽습니다. '백일'은 10진수의 자리값을 붙여 읽는 말이기 때문입니다. $101_2 = 1 \times 4 + 0 \times 2 + 1 \times 1 = 5$ 입니다.

8진수(octal number)

기수를 8로 하는 수 체계로, 0에서 7 사이의 숫자를 이용해 수를 나타냅니다. 자리값은 8^i 입니다.

16진수(hexadecimal number)

기수를 16으로 하는 수 체계로, 0에서 9와 A(10)에서 F(15) 사이의 숫자를 이용해 수를 나타냅니다. 자리값은 16^i 입니다.

16진수는 한 자리에 0부터 15까지 열여섯 가지 숫자가 필요합니다. 10부터 15까지는 한 글자로 적을 숫자가 없으므로 A, B, C, D, E, F를 빌려 씁니다. A는 10, B는 11, C는 12, D는 13, E는 14, F는 15입니다. 16진수의 10은 '십'이 아니라 16을 뜻합니다($10_{16} = 1 \times 16 + 0 = 16$). 기수를 밝히지 않으면 같은 10이 2, 8, 10, 16 가운데 무엇인지 알 수 없으므로, 헷갈릴 수 있는 곳에서는 아래 첨자로 기수를 붙입니다. 표 4-2는 0부터 16까지를 네 수 체계로 적은 것입니다.

10진수	2진수	8진수	16진수	10진수	2진수	8진수	16진수
0	0000	0	0	9	1001	11	9
1	0001	1	1	10	1010	12	A
2	0010	2	2	11	1011	13	B
3	0011	3	3	12	1100	14	C
4	0100	4	4	13	1101	15	D
5	0101	5	5	14	1110	16	E
6	0110	6	6	15	1111	17	F
7	0111	7	7	16	10000	20	10
8	1000	10	8				

표 4-2 0부터 16까지의 네 수 체계. 2진수는 네 자리로 맞추어 적었습니다.

표에서 16진수 한 자리가 2진수 네 자리와 꼭 맞게 짝지어지는 데 주목하세요. 2진수 네 자리로 나타낼 수 있는 수는 0000부터 1111까지 열여섯 가지, 곧 0부터 15까지이고, 이것이 16진수 한 자리가 나타내는 범위와 같습니다. 16에 이르면 2진수도 다섯째 자리로, 16진수도 둘째 자리로 넘어갑니다. 8진수 한 자리는 같은 이유로 2진수 세 자리와 짝지어집니다. 2-2)의 변환이 이 짝을 씁니다.

예제 4-15

9B.FE3₁₆을 기수와 자리수를 이용해 풀어 쓰시오.

풀이

소수점 왼쪽의 9와 B는 16¹과 16⁰의 자리이고, 오른쪽의 F, E, 3은 16⁻¹, 16⁻², 16⁻³의 자리입니다. 소수점에서 멀어질수록 지수가 하나씩 작아지므로 마지막 3의 자리는 16⁻³입니다. 첫 줄처럼 쓴 뒤 숫자 B, F, E를 11, 15, 14로 바꾸면 둘째 줄이 됩니다.

$$\begin{aligned}
 9B.FE3_{16} &= 9 \times 16^1 + B \times 16^0 + F \times 16^{-1} + E \times 16^{-2} + 3 \times 16^{-3} \\
 &= 9 \times 16^1 + 11 \times 16^0 + 15 \times 16^{-1} + 14 \times 16^{-2} + 3 \times 16^{-3}
 \end{aligned}$$

계산하면 $144 + 11 + 0.9375 + 0.0546875 + 0.000732421875 = 155.992919921875$ 입니다. 이렇게 자리값을 곱해 더하는 것이 곧 16진수를 10진수로 바꾸는 방법입니다.

2-1) 수 체계별 사칙연산

어느 수 체계에서든 사칙연산의 방법은 10진수와 같습니다. 자리를 맞춰 아래 자리부터 계산하고, 한 자리에 담을 수 없는 값이 되면 윗자리로 올리며, 뺄 수 없으면 윗자리에서 빌려 옵니다. 다른 것은

올리고 빌리는 단위뿐입니다. 10진수에서는 10이 되면 1을 올리고 빌려 올 때 10을 받지만, 2진수에서는 2, 16진수에서는 16이 그 단위입니다.

예제 4-17

다음 2진수를 연산하시오. $101 + 11$, $100 - 11$, 1101×11 , $10110 \div 101$

풀이

덧셈 맨 아래 자리부터 더합니다. $1 + 1 = 2$ 인데 2진수 한 자리에는 2를 담을 수 없으므로 0을 쓰고 1을 올립니다. 둘째 자리는 $0 + 1$ 에 올라온 1을 더해 다시 2이므로 0을 쓰고 1을 올립니다. 셋째 자리도 1에 올라온 1을 더해 2이므로 0을 쓰고 1을 올려, 맨 앞에 1이 생깁니다.

$$\begin{array}{r} 101 \\ + 11 \\ \hline 1000 \end{array}$$

10진수로 확인하면 $5 + 3 = 8$ 입니다.

뺄셈 맨 아래 자리 0에서 1을 뺄 수 없으므로 윗자리에서 빌려 와야 하는데, 둘째 자리도 0이라 빌려줄 것이 없습니다. 그래서 셋째 자리의 1을 빌립니다. 셋째 자리의 1은 둘째 자리에서 보면 2입니다. 그 가운데 1을 둘째 자리에 남기고 1을 맨 아래 자리로 내려보내면, 맨 아래 자리에서는 2가 됩니다. 맨 아래 자리는 $2 - 1 = 1$, 둘째 자리는 남은 1에서 1을 빼 0, 셋째 자리는 빌려주고 0이 됩니다.

$$\begin{array}{r} 100 \\ - 11 \\ \hline 1 \end{array}$$

결과는 001, 곧 1입니다. 10진수로 $4 - 3 = 1$ 입니다.

곱셈 2진수의 곱셈은 오히려 10진수보다 쉽습니다. 곱하는 수의 각 자리가 0 아니면 1이므로, 부분곱은 0이거나 곱해지는 수를 그대로 옮겨 적은 것입니다. 1101×11 에서 아래 자리 1을 곱하면 1101이고, 윗자리 1을 곱하면 1101을 한 자리 왼쪽으로 민 것입니다. 이 둘을 더합니다.

$$\begin{array}{r} 1101 \\ \times 11 \\ \hline 1101 \\ + 1101 \\ \hline 100111 \end{array}$$

10진수로 $13 \times 3 = 39$ 이고, $100111_2 = 32 + 4 + 2 + 1 = 39$ 입니다. 곱하는 수에 0인 자리가 있으면 그 줄의 부분곱은 0이 됩니다. 예를 들어 1101×101 은 1101, 0000, 1101을 한 자리씩 밀어 더해 $1000001(13 \times 5 = 65)$ 입니다. 이처럼 2진수 곱셈은 옮겨 적기, 밀기, 더하기만으로 되므로 컴퓨터의 회로로 간단히 만들 수 있습니다.

나눗셈 10진수의 긴 나눗셈과 같은 방법입니다. 10110의 앞 세 자리 101에서 나누는 수 101을 한 번 뺄 수 있으므로 몫의 첫 자리에 1을 쓰고 빼면 0이 남습니다. 다음 자리 1을 내려 01을 만들면 101보다 작으니 몫에 0을 쓰고, 다시 0을 내려 010을 만들어도 101보다 작으니 몫에 0을 씁니다. 몫은 100, 나머지는 10입니다.

$$\begin{array}{r} 100 \\ 101 \overline{) 10110} \\ \underline{101} \\ 10 \end{array}$$

10진수로 $22 = 5 \times 4 + 2$ 입니다. 1절의 나머지 연산으로 쓰면 $10110_2 \bmod 101_2 = 10_2$ 입니다.

$\therefore 101 + 11 = 1000, 100 - 11 = 1, 1101 \times 11 = 100111, 10110 \div 101 = 100(\text{나머지 } 10)$

예제 4-19

다음 16진수를 연산하십시오. $939 + 99$, $5A4 - CE$, $D82 \times 39$, $BB1 \div 3C$

풀이

각 자리를 10진수로 생각해 계산한 뒤, 16 이상이 되면 16을 단위로 올리고 결과를 다시 16진수 숫자로 적습니다.

덧셈 맨 아래 자리는 $9 + 9 = 18$ 이고 16을 넘으므로 $18 - 16 = 2$ 를 쓰고 1을 올립니다. 둘째 자리는 $3 + 9 + 1 = 13$ 이고 13은 D입니다. 셋째 자리는 9를 그대로 내려 9D2입니다.

$$\begin{array}{r} 939 \\ + 99 \\ \hline 9D2 \end{array}$$

뺄셈 맨 아래 자리 4에서 E(14)를 뺄 수 없으므로 윗자리에서 16을 빌려 옵니다. $20 - 14 = 6$ 인데, 빌려 온 16에서 먼저 14를 빼 2를 얻고 원래 있던 4를 더하면 암산이 쉽습니다. 둘째 자리 A(10)는 1을 빌려주어 9가 되었고, 9에서 C(12)를 뺄 수 없으니 다시 16을 빌려 옵니다. $16 - 12 = 4$ 에 9를 더하면 13, 곧 D입니다. 셋째 자리 5는 1을 빌려주어 4입니다.

$$\begin{array}{r} 5A4 \\ - CE \\ \hline 4D6 \end{array}$$

곱셈 먼저 $D82$ 에 9를 곱합니다. $2 \times 9 = 18 = 16 + 2$ 이므로 2를 쓰고 1을 올립니다. $8 \times 9 = 72$ 에 올라온 1을 더하면 $73 = 16 \times 4 + 9$ 이므로 9를 쓰고 4를 올립니다. $D \times 9 = 13 \times 9 = 117$ 에 4를 더하면 $121 = 16 \times 7 + 9$ 이므로 9를 쓰고 7을 올립니다. 그래서 첫 부분곱은 7992입니다. 같은 방법으로 $D82 \times 3 = 2886$ 이고, 이것을 한 자리 왼쪽으로 밀어 더합니다. 더할 때도 $9 + 6 = 15$ 는 F, $9 + 8 = 17$ 은 1을 쓰고 1을 올리는 식으로 16을 단위로 올립니다.

$$\begin{array}{r} D82 \\ \times 39 \\ \hline 7992 \\ + 2886 \\ \hline 301F2 \end{array}$$

나눗셈 BB1의 앞 두 자리 BB를 3C로 나눕니다. $3C \times 3 = B4$ 이고 $3C \times 4 = F0$ 은 BB보다 크므로 몫의 첫 자리는 3입니다. $BB - B4 = 7$ 이고, 다음 자리 1을 내려 71을 만듭니다. $3C \times 1 = 3C$ 이고 $3C \times 2 = 78$ 은 71보다 크므로 몫의 다음 자리는 1이며, $71 - 3C = 35$ 가 나머집니다.

$$\begin{array}{r} 31 \\ 3C \overline{) BB1} \\ \underline{B4} \\ 71 \\ \underline{3C} \\ 35 \end{array}$$

계산이 낫설면 10진수로 바꾸어 확인합니다. 예를 들어 $D82_{16} = 3458$, $39_{16} = 57$ 이고 $3458 \times 57 = 197106$ 인데, 이것을 16진수로 적은 것이 $301F2_{16}$ 입니다.

$\therefore 939 + 99 = 9D2$, $5A4 - CE = 4D6$, $D82 \times 39 = 301F2$, $BB1 \div 3C = 31$ (나머지 35)

2-2) 수 체계의 변환

수 체계의 변환은 같은 수를 다른 기수의 자리값으로 다시 나누어 적는 일입니다. 다른 수 체계를 10진수로 바꾸기는 쉽습니다. 예제 4-15처럼 각 자리의 숫자에 자리값을 곱해 더하면 됩니다. 이 소절에서는 반대 방향, 곧 10진수를 2진수로 바꾸는 방법과 2진수를 8진수와 16진수로 바꾸는 방법을 봅니다. 10진수의 정수부와 소수부는 바꾸는 방법이 다르므로 나누어 다룹니다.

예제 4-20

10진수 163.875를 2진수로 변환하시오.

풀이

정수부: 2로 거듭 나누기 163을 2진수로 바꾼다는 것은 $163 = a_k 2^k + \dots + a_1 2^1 + a_0 2^0$ 인 숫자 a_i (0 또는 1)를 찾는 일입니다. 오른쪽 식에서 a_0 을 뺀 나머지 항은 모두 2의 배수이므로, 163을 2로 나눈 나머지가 바로 a_0 입니다. $163 = 2 \times 81 + 1$ 이므로 $a_0 = 1$, 1절의 기호로 $a_0 = 163 \bmod 2$ 입니다. 몫 81은 $a_k 2^{k-1} + \dots + a_2 2^1 + a_1$ 이므로, 81을 다시 2로 나눈 나머지가 a_1 입니다. 이렇게 몫이 0이 될 때까지 2로 거듭 나누면 나머지가 $a_0, a_1, a_2, \dots$ 의 순서로 나옵니다. 표 4-3이 그 과정입니다.

나눌 수	2로 나눈 몫	나머지	자리
163	81	1	a_0 (최하위 자리)
81	40	1	a_1
40	20	0	a_2
20	10	0	a_3
10	5	0	a_4
5	2	1	a_5
2	1	0	a_6
1	0	1	a_7 (최상위 자리)

표 4-3 163을 2로 거듭 나눈 몫과 나머지

먼저 나온 나머지가 최하위 자리이고 마지막에 나온 나머지가 최상위 자리입니다. 그래서 답을 적을 때는 나머지를 아래에서 위로 거슬러 읽어 $163 = 10100011_2$ 입니다. 1인 자리의 자리값을 더해 확인하면 $2^7 + 2^5 + 2^1 + 2^0 = 128 + 32 + 2 + 1 = 163$ 입니다.

소수부: 2를 거듭 곱하기 소수부 0.875는 $0.875 = a_{-1}2^{-1} + a_{-2}2^{-2} + a_{-3}2^{-3} + \dots$ 인 숫자를 찾아야 합니다. 이번에는 2를 곱합니다. 양변에 2를 곱하면 $1.75 = a_{-1} + a_{-2}2^{-1} + a_{-3}2^{-2} + \dots$ 이 됩니다. 오른쪽에서 a_{-1} 을 뺀 나머지 항은 모두 합해도 1보다 작으므로, 곱한 결과의 정수부가 바로 a_{-1} 입니다. $a_{-1} = 1$ 이고, 남은 소수부 0.75에 다시 2를 곱하면 그 정수부가 a_{-2} 가 됩니다. 이렇게 소수부가 0이 될 때까지 2를 거듭 곱하면 정수부가 $a_{-1}, a_{-2}, \dots$ 의 순서로 나옵니다. 표 4-4가 그 과정입니다.

곱할 소수부	2를 곱한 결과	정수부	자리
0.875	1.75	1	a_{-1} (소수점 바로 아래)
0.75	1.5	1	a_{-2}
0.5	1.0	1	a_{-3}

표 4-4 0.875에 2를 거듭 곱한 결과

이번에는 먼저 나온 정수부가 소수점 바로 아래 자리이므로 위에서 아래로 읽어 $0.875 = 0.111_2$ 입니다. $2^{-1} + 2^{-2} + 2^{-3} = 0.5 + 0.25 + 0.125 = 0.875$ 로 확인됩니다.

$$\therefore 163.875_{10} = 10100011.111_2$$

정수부는 나머지를 아래에서 위로, 소수부는 정수부를 위에서 아래로 읽습니다. 읽는 방향이 반대라서 헷갈리기 쉬운데, 둘 다 '처음 나온 숫자가 소수점에 가장 가까운 자리'라고 기억하면 하나로 묶입니다. 정수부의 첫 나머지는 소수점 바로 왼쪽 a_0 이고, 소수부의 첫 정수부는 소수점 바로 오른쪽 a_{-1} 입니다. 그 뒤로는 소수점에서 바깥쪽으로 한 자리씩 나옵니다.

예제

10진수 14.75를 2진수로 변환하시오.

풀이

정수부 14를 2로 거듭 나누면 $14 = 2 \times 7 + 0$, $7 = 2 \times 3 + 1$, $3 = 2 \times 1 + 1$, $1 = 2 \times 0 + 1$ 이므로 나머지는 차례로 0, 1, 1, 1입니다. 아래에서 위로 읽어 $14 = 1110_2$ 입니다. 나눈 과정을 식으로 모아 보면 나머지가 왜 자리의 숫자가 되는지 다시 보입니다. 몫을 거듭 풀어 쓰면 다음과 같습니다.

$$\begin{aligned}14 &= 7 \times 2^1 + 0 \times 2^0 \\&= (3 \times 2 + 1) \times 2^1 + 0 \times 2^0 = 3 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\&= (1 \times 2 + 1) \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\&= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0\end{aligned}$$

소수부 0.75는 $0.75 \times 2 = 1.5$ 에서 1, $0.5 \times 2 = 1.0$ 에서 1을 얻어 $0.75 = 0.11_2$ 입니다. $0.75 = 0.5 + 0.25 = 1 \times 2^{-1} + 1 \times 2^{-2}$ 이기 때문입니다.

$$\therefore 14.75_{10} = 1110.11_2$$

소수부에 2를 거듭 곱하는 과정이 늘 끝나는 것은 아닙니다. 0.1에 2를 거듭 곱하면 0.2, 0.4, 0.8, 1.6, 1.2가 되어 정수부가 0, 0, 0, 1, 1입니다. 1.2의 소수부 0.2는 처음에 나온 0.2와 같으므로, 그 뒤로는 0.4, 0.8, 1.6, 1.2가 끝없이 되풀이됩니다. 그래서 $0.1_{10} = (0.0001100110011 \dots)_2$ 로 0011이 끝없이 반복됩니다. 10진수로 끝나는 소수도 2진수로 끝나지 않을 수 있는 것입니다. 2진수의 소수부가 끝나려면 그 수가 정수를 2^l 로 나눈 꼴이어야 하는데, $0.1 = \frac{1}{10}$ 은 분모에 5가 있어 그렇게 쓸 수 없습니다. 비트 수가 정해진 컴퓨터는 이런 수를 어느 자리에서 잘라 저장하므로 작은 오차가 생깁니다.

2진수를 8진수와 16진수로

10진수를 8진수나 16진수로 바꿀 때도 8이나 16으로 거듭 나누고 곱하는 같은 방법을 쓸 수 있지만, 계산이 번거롭습니다. 먼저 2진수로 바꾼 뒤 2진수를 묶어 바꾸는 편이 훨씬 간편합니다. $2^3 = 8$ 이므로 2진수 세 자리는 8진수 한 자리가 되고, $2^4 = 16$ 이므로 2진수 네 자리는 16진수 한 자리가 됩니다. 표 4-2에서 본 짝입니다. 예를 들어 세 자리씩 묶으면 $a_5 2^5 + a_4 2^4 + a_3 2^3 = (a_5 2^2 + a_4 2^1 + a_3) \times 8^1$ 처럼, 묶음 하나가 8의 거듭제곱 하나에 곱해지는 0부터 7까지의 숫자가 됩니다.

묶는 기준은 소수점입니다. 소수점에서 시작해 정수부는 왼쪽으로, 소수부는 오른쪽으로 세 자리(8진수)나 네 자리(16진수)씩 묶습니다. 끝에 자리가 모자라면 0을 채우는데, 정수부는 맨 왼쪽에, 소수부는 맨 오른쪽에 채웁니다. 정수부의 왼쪽에 붙인 0은 값을 바꾸지 않습니다($011 = 11$). 소수부도 오른쪽에 붙인 0은 값을 바꾸지 않지만($0.111 = 0.1110$), 왼쪽에 붙이면 0.0111 이 되어 다른 수가 됩니다. 0을 채우는 쪽이 정수부와 소수부에서 반대인 까닭입니다.

예제 4-20

10진수 163.875를 8진수와 16진수로 변환하시오.

풀이

앞에서 구한 $163.875_{10} = 10100011.111_2$ 를 씁니다. 8진수로는 소수점에서부터 세 자리씩 묶습니다. 정수부 10100011은 여덟 자리라 맨 왼쪽 묶음이 두 자리뿐이므로 앞에 0을 하나 채웁니다.

$$\underbrace{010}_2 \underbrace{100}_4 \underbrace{011}_3 . \underbrace{111}_7$$

묶음마다 자리값 4, 2, 1로 읽으면 010은 2, 100은 4, 011은 3, 111은 7이므로 $163.875_{10} = 243.7_8$ 입니다.

16진수로는 네 자리씩 묶습니다. 정수부 10100011은 마침 네 자리씩 둘로 나뉘고, 소수부 111은 세 자리라 오른쪽에 0을 채워 1110으로 만듭니다.

$$\underbrace{1010}_A \underbrace{0011}_3 . \underbrace{1110}_E$$

묶음마다 자리값 8, 4, 2, 1로 읽으면 1010은 10이므로 A, 0011은 3, 1110은 14이므로 E입니다.

$$\therefore 163.875_{10} = 243.7_8 = A3.E_{16}$$

거꾸로 8진수나 16진수를 2진수로 바꿀 때는 각 자리를 세 자리나 네 자리 2진수로 풀어 늘어놓으면 됩니다. 10진수로 바꿀 때는 예제 4-15처럼 자리값을 곱해 더합니다. $A3.E_{16} = 10 \times 16^1 + 3 \times 16^0 + 14 \times 16^{-1} = 160 + 3 + 0.875 = 163.875$ 이고, 2진수에서도 $10100011.111_2 = 2^7 + 2^5 + 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} = 163.875$ 입니다. 8진수는 기수를 8로 하여 같은 방법을 씁니다.

3. 보수의 표현

지금까지는 수를 적을 때 자리 수에 제한이 없었고, 음수는 앞에 -를 붙여 적었습니다. 컴퓨터는 사정이 다릅니다. 수 하나를 담는 비트의 개수가 정해져 있고, 담을 수 있는 것은 0과 1뿐이라 - 기호를 따로 적을 자리가 없습니다. 그래서 정해진 n 비트 안에서 음수를 나타내는 약속이 필요합니다. 이 절의 보수 표현이 그 약속이고, 그 바탕이 되는 것이 보수입니다.

보수(complement)

어떤 수에 더해서 기준이 되는 수를 채우는 수를 그 기준에 대한 보수라고 합니다. 예를 들어 $3 + n = 7$ 에서 $n = 4$ 이므로, 3에 대한 7의 보수는 4입니다.

보수는 채워 주는 수입니다. 영어 이름 complement는 3장의 여집합(보집합)과 같은 낱말인데, 전체에서 A 를 빼고 남은 부분이 여집합이듯 기준에서 어떤 수를 빼고 남은 만큼이 보수입니다. 2진수에서는 두 가지 보수를 씁니다.

1의 보수(1's complement)

2진수의 각 자리에 더해서 1이 되는 수, 곧 각 비트를 0은 1로, 1은 0으로 바꾼 수를 1의 보수라고 합니다. 예: 2진수 00110101의 1의 보수는 11001010입니다.

2의 보수(2's complement)

n 비트 2진수에 더해서 2^n 이 되는 수, 곧 1의 보수에 1을 더한 수를 2의 보수라고 합니다. 예: 2진수 00110101의 2의 보수는 $11001010 + 1 = 11001011$ 입니다.

두 보수의 기준을 n 비트 전체로 보면 뜻이 분명해집니다. 8비트 수 00110101에 1의 보수 11001010을 더하면 모든 자리가 1인 11111111, 곧 $2^8 - 1 = 255$ 가 됩니다. 자리마다 1을 채우기 때문입니다. 2의 보수 11001011은 이보다 1이 크므로, 더하면 $2^8 = 256$, 2진수로 100000000이 됩니다. 아홉 자리이므로 8비트 안에는 00000000만 남습니다.

$$00110101 + 11001010 = 11111111 = 2^8 - 1$$

$$00110101 + 11001011 = 100000000 = 2^8$$

두 이름도 여기서 왔습니다. 기수가 r 인 수 체계에서 n 자리 수 x 에 대해 $r^n - x$ 를 r 의 보수, $(r^n - 1) - x$ 를 $r - 1$ 의 보수라고 부릅니다. 2진수에서는 $r = 2$ 이므로 2의 보수와 1의 보수가 되고, 10진수에서는 10의 보수와 9의 보수가 됩니다. 10진수 한 자리로 보면 5, 4, 3의 9의 보수는 9를 채우는 4, 5, 6이고, 10의 보수는 10을 채우는 5, 6, 7입니다. 9의 보수에 1을 더하면 10의 보수가 되는 것도 1의 보수와 2의 보수의 관계와 같습니다.

보수는 자리 수에 따라 달라진다는 점에 주의하세요. 4비트에서 0010의 2의 보수는 1110이지만, 8비트에서 00000010의 2의 보수는 11111110입니다. 기준 2^n 이 비트 수 n 으로 정해지기 때문입니다. 10진수에서도 3의 10의 보수는 한 자리로 보면 7이지만, 두 자리로 보면 $100 - 3 = 97$ 입니다.

2의 보수를 구하는 요령이 있습니다. 최하위 비트부터 왼쪽으로 가면서 처음 나오는 1까지는 그대로 적고, 그 왼쪽의 비트는 모두 뒤집습니다. 00110100이면 오른쪽 끝의 100까지는 그대로 두고 앞의 00110을 뒤집어 11001을 적으므로, 2의 보수는 11001100입니다. 1의 보수를 만든 뒤 1을 더해 보면 이 요령이 맞는 까닭이 보입니다. 00110100의 1의 보수는 11001011입니다. 오른쪽 끝의 연속된 1에 1을 더하면 그 자리들이 모두 0이 되면서 올림이 왼쪽으로 번지다가, 처음 만나는 0을 1로 바꾸고 멈춥니다. 그 0은 원래 수에서 처음 나오는 1이 있던 자리입니다. 그래서 원래 수의 오른쪽 끝 0들과 처음 나오는 1은 원래대로 돌아오고, 그보다 왼쪽은 1의 보수 그대로 뒤집힌 채 남습니다.

3-1) 컴퓨터에서의 데이터 표현

부호화-절대치 표현

부호화-절대치 표현(sign-magnitude representation)

n 비트의 최상위 비트를 부호 비트로 쓰고, 나머지 $n - 1$ 비트에 데이터의 절댓값을 그대로 나타내는 방법입니다. 부호 비트가 0이면 양수, 1이면 음수입니다.

가장 왼쪽 비트, 곧 자리값이 가장 큰 비트를 최상위 비트(MSB, most significant bit)라 하고, 가장 오른쪽 비트를 최하위 비트(LSB, least significant bit)라고 합니다. 부호화-절대치 표현은 종이에 -53이라고 적는 방식을 그대로 옮긴 것입니다. - 기호 자리에 비트 하나를 쓰고, 나머지 비트에 53을 2진수로 적습니다. 부호 비트를 빼 나머지 비트를 절대치 비트 또는 데이터 비트라고 합니다.

예제 4-28

10진수 +53과 -53에 대하여 8bit 부호화-절대치 표현으로 나타내시오.

풀이

먼저 절댓값 53을 2진수로 바꿉니다. $53 = 32 + 16 + 4 + 1$ 이므로 $|53_{10}| = 110101_2$ 입니다. 절대치 비트는 7비트이므로 앞에 0을 채워 0110101로 맞춥니다. +53은 부호 비트 0을 앞에 붙여 00110101이고, -53은 부호 비트 1을 붙여 10110101입니다. 두 수는 부호 비트 하나만 다릅니다.

$$\therefore +53_{10} = 00110101_2, -53_{10} = 10110101_2$$

그림 4-2는 예제 4-28의 두 수를 비트 칸에 나타낸 것입니다. 칠한 칸이 부호 비트입니다.



그림 4-2 8비트 부호화-절대치 표현으로 나타낸 +53과 -53

1의 보수 표현과 2의 보수 표현

보수 표현은 부호화-절대치 표현에서 출발해 음수만 바꾼 것입니다. 1의 보수 표현과 2의 보수 표현은 두 가지가 같습니다. 첫째, 음수 표현에만 적용합니다. 양수는 부호화-절대치 표현과 똑같이 나타냅니다. 둘째, 음수의 부호화-절대치 표현에서 부호 비트 1은 그대로 씁니다. 다른 점은 음수의 절대치 비트를 바꾸는 방법입니다. 1의 보수 표현은 절대치 비트를 1의 보수로(0은 1로, 1은 0으로) 바꾸고, 2의 보수 표현은 절대치 비트를 2의 보수로(1의 보수에 1을 더해) 바꿉니다.

예제

+53과 -53을 8bit 1의 보수 표현과 2의 보수 표현으로 나타내시오.

풀이

+53은 양수이므로 두 표현 모두 부호화-절대치 표현과 같은 00110101입니다. -53은 부호 비트 1을 두고, 부호 비트를 뺀 7비트 데이터 비트 0110101₂를 바꿉니다. 1의 보수 표현은 0110101을 뒤집은 1001010을 붙여 11001010이고, 2의 보수 표현은 여기에 1을 더해 11001011입니다.

∴ +53은 두 표현 모두 00110101이고, -53은 1의 보수 표현으로 11001010, 2의 보수 표현으로 11001011입니다.

결과를 보수의 정의와 견주어 보세요. -53의 1의 보수 표현 11001010은 +53의 8비트 00110101을 통째로 뒤집은 것과 같고, 2의 보수 표현 11001011은 00110101의 2의 보수와 같습니다. +53의 부호 비트 0을 뒤집으면 음수의 부호 비트 1이 되므로, '부호 비트를 1로 두고 절대치 비트만 바꾸기'와 '+53의 n 비트 전체를 보수로 바꾸기'가 같은 결과를 내는 것입니다. 2의 보수에서도 같은 결과가 나오는데, 1을 더할 때 올림이 부호 비트까지 번지는 경우는 절댓값이 0일 때뿐이기 때문입니다. 그래서 보수 표현에서는 음수 $-m$ 을 $+m$ 의 보수로 나타낸다고 말해도 됩니다.

데이터의 범위

n 비트로 나타낼 수 있는 수의 범위는 표현마다 다릅니다. 1의 보수 표현은 $-(2^{n-1} - 1)$ 부터 $2^{n-1} - 1$ 까지이고, 2의 보수 표현은 -2^{n-1} 부터 $2^{n-1} - 1$ 까지입니다. 부호화-절대치 표현의 범위는 1의 보수 표현과 같습니다.

가장 큰 양수부터 봅시다. 양수는 세 표현 모두 부호 비트가 0이고 나머지 $n - 1$ 비트에 절댓값을 적으므로, 가장 큰 양수는 $n - 1$ 비트가 모두 1인 수입니다. 4비트라면 0111, 곧 7입니다. $n - 1$ 비트가 모두 1인 수는 $2^{n-1} - 1$ 입니다. 111에 1을 더하면 1000이 되듯이, 모두 1인 $n - 1$ 자리 수에 1을 더하면 그 바로 위 자리값 2^{n-1} 이 되기 때문입니다. 4비트에서 $2^3 - 1 = 7$, 8비트에서 $2^7 - 1 = 127$ 입니다.

가장 작은 음수는 표현마다 다릅니다. 표 4-5는 4비트 비트열 16개가 세 표현에서 각각 어떤 수인지 나타낸 것입니다.

비트열	부호화-절대치 표현	1의 보수 표현	2의 보수 표현
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	-0	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0	-1

표 4-5 4비트 비트열이 세 표현에서 나타내는 수. 칠한 칸이 -0과 -8입니다.

1의 보수 표현에서는 0000이 +0이고, 그 1의 보수 1111이 -0입니다. 0을 나타내는 비트열이 둘이므로 16개 비트열로 15개 수, -7부터 7까지만 나타냅니다. 부호화-절대치 표현도 1000이 -0이어서 사정이 같습니다. 2의 보수 표현에서는 0000의 2의 보수를 구하면 $1111 + 1 = 10000$ 이 되어 4비트 안에는 다시 0000이 남습니다. 0이 비트열 하나로만 나타나는 것입니다. 남은 비트열 1000이 -8을 나타내므로, 2의 보수 표현은 음수가 하나 더 많은 -8부터 7까지를 나타냅니다. 1000이 -8인 것은 -7을 나타내는 1001보다 하나 작은 비트열이기 때문입니다. 8비트라면 1의 보수 표현은 -127부터 127까지, 2의 보수 표현은 -128부터 127까지입니다.

초과

초과(overflow)

연산 결과가 그 표현이 나타낼 수 있는 데이터의 범위를 벗어나는 경우를 초과라고 합니다.

초과가 일어나면 결과가 n 비트에 맞지 않아 엉뚱한 수가 남습니다. 초과는 다음 두 경우에 드러납니다. 첫째, 두 양수를 더했는데 음수가 나오는 경우입니다. 둘째, 두 음수를 더했는데 양수가 나오는 경우입니다. 4비트에서 가장 큰 양수 0111에 1만 더해도 1000이 되어 부호 비트가 1로

바뀌는 것을 떠올리면 됩니다. 부호가 다른 두 수를 더할 때는 초과가 일어나지 않습니다. 합이 두 수 사이에 있으므로 두 수가 범위 안이면 합도 범위 안이기 때문입니다.

예제

4bit 부호화-절대치 표현에서 다음을 계산하시오. ① $4 + 5$ ② $(-3) + (-4)$

풀이

① $4 = 0100$ 과 $5 = 0101$ 을 2진수처럼 더하면 1001 입니다. 부호화-절대치 표현으로 읽으면 부호 비트가 1이고 절댓값이 001 이므로 -1 입니다. 참값 9가 4비트 부호화-절대치 표현의 범위 -7 부터 7 을 벗어나 초과가 일어났고, 두 양수의 합이 음수로 나왔습니다.

$$4_{10} + 5_{10} = 0100_2 + 0101_2 = 1001_2 = -1_{10} \neq 9_{10}$$

② $-3 = 1011$ 과 $-4 = 1100$ 을 더하면 10111 로 다섯 자리가 됩니다. 4비트에 남은 0111 을 읽으면 $+7$ 이므로, 두 음수의 합이 양수로 나왔습니다.

$$(-3) + (-4) = 1011 + 1100 = 10111 \rightarrow 0111 = 7 \neq -7$$

그런데 ②의 참값 -7 은 범위 안에 있습니다. 4비트 부호화-절대치 표현으로 1111 입니다. 범위를 넘지 않았는데도 결과가 틀린 것은 부호화-절대치 표현의 비트열을 보통 2진수처럼 더했기 때문입니다. 부호 비트는 크기를 나타내는 자리가 아닌데, 덧셈은 그 자리도 자리값이 있는 것처럼 더해 버립니다. 부호화-절대치 표현으로 바르게 계산하려면 두 수의 부호를 먼저 비교해, 같으면 절댓값을 더하고 다르면 큰 절댓값에서 작은 절댓값을 빼는 별도의 과정을 거쳐야 합니다. 그래서 결과의 부호가 바뀌는 것으로 초과를 알아보는 방법은 덧셈이 바르게 되는 표현에서만 믿을 수 있습니다. 4절에서 2의 보수 표현으로 다시 확인합니다.

3-2) 보수 표현을 사용하는 이유

3-1)에서 본 세 표현의 한계를 정리하면 다음과 같습니다.

- **부호화-절대치 표현:** 연산의 결과가 부정확합니다. 4비트에서 $4 + 5$ 처럼 초과가 일어날 때는 물론, $(-3) + (-4)$ 처럼 범위 안의 계산도 비트열을 그대로 더하면 틀립니다. 또 0을 $0000(+0)$ 과 $1000(-0)$ 두 가지로 나타냅니다.
- **1의 보수 표현:** 연산을 정확하게 하려면 덧셈 뒤에 한 단계를 더 거쳐야 합니다(4절에서 설명합니다). 0도 $0000(+0)$ 과 $1111(-0)$ 두 가지로 나타냅니다.
- **2의 보수 표현:** 위의 두 문제를 모두 해결합니다. 덧셈만으로 연산이 정확하고, 0을 0000 하나로 나타냅니다.

0의 표현이 둘이면 비트열 하나를 낭비하고, 두 수가 같은지 비교할 때 $+0$ 과 -0 을 같은 수로 다루는 처리가 따로 필요합니다.

뺄셈을 덧셈으로

보수를 쓰는 더 근본적인 까닭은 뺄셈에 있습니다. 컴퓨터는 덧셈 회로로 계산합니다. 뺄셈을 위한 회로를 따로 두는 대신 빼는 수를 보수로 바꾸어 더하면, 덧셈 회로 하나로 뺄셈까지 할 수 있습니다. 보수를 만드는 일도 비트를 뒤집는 것이어서 회로로 간단히 만들 수 있습니다. 넓게 보면 보수는 컴퓨터 안에서 음수를 나타내는 방법이고, 음수를 이렇게 나타내 두면 $x - y$ 는 $x + (-y)$ 라는 덧셈이 됩니다.

보수를 더하는 것이 왜 빼는 것과 같은지 10진수로 먼저 봅시다. 칸이 10개인 막대를 생각하고 $5 - 3$ 을 계산해 봅시다. 5에서 3을 빼는 대신, 3의 10의 보수 7을 5에 더합니다. 그림 4-3처럼 5칸을 채운 막대에 7칸을 더 채우면 10칸이 다 차고 2칸이 넘칩니다. $5 + 7 = 12$ 입니다. 다 찬 10칸을 버리면 남은 2칸이 남고, 이것이 $5 - 3 = 2$ 입니다. 3을 빼는 대신 3과 합해 10이 되는 7을 더했으니, 10을 버리면 3을 뺀 것과 같아지는 것입니다.

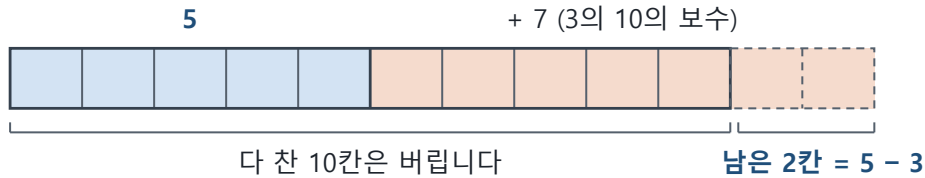


그림 4-3 10칸 막대로 본 $5 - 3 = 5 + 7 - 10$

식으로 쓰면 다음과 같습니다.

$$x - y = x + (10 - y) - 10$$

$10 - y$ 가 y 의 10의 보수이고, 마지막의 -10 이 다 찬 10칸을 버리는 일입니다. $+10$ 과 -10 이 서로 지워지므로 식은 처음의 $x - y$ 로 돌아옵니다. 보수를 더한 뒤 넘친 기준만큼을 버리면 뺄셈과 같아지는 것입니다.

2진수에서도 원리는 같습니다. n 비트에서는 기준이 2^n 이고, $2^n - y$ 가 y 의 2의 보수입니다.

$$x - y = x + (2^n - y) - 2^n$$

2^n 은 n 비트 바로 위 자리의 1이므로, 2^n 을 버린다는 것은 n 비트 밖으로 넘친 자리올림을 버린다는 뜻입니다. 2진수에 맞는 보수는 2의 보수이고, 10진수에 맞는 보수는 10의 보수입니다. 기수와 보수가 이렇게 짝을 이룰 때 '보수를 더하고 넘친 것을 버린다'가 그대로 통합니다.

짝이 맞지 않으면 이 비유는 끊깁니다. 10진수에서 9의 보수를 쓰면 $5 - 3 = 5 + 6 = 11$ 인데, 10을 버리면 1이 남아 답 2보다 1이 모자랍니다. 9의 보수는 9를 채우는 수인데, 넘칠 때는 10을 채워야 윗자리로 넘어가기 때문입니다. 2진수의 1의 보수도 $2^n - 1$ 을 채우는 수라서 같은 일이 생깁니다. 이 모자란 1을 어떻게 보충하는지가 4절에서 다룰 1의 보수 연산의 핵심입니다.

3-3) 보수의 10진수 변환

1의 보수 표현이나 2의 보수 표현으로 저장된 음수를 10진수로 바꾸는 방법은 두 가지입니다. 부호 비트가 0이면 양수이므로 그냥 2진수로 읽으면 되고, 아래 방법은 부호 비트가 1일 때 씁니다.

- **방법 ①** 그 표현으로 나타낼 수 있는 가장 작은 수에, 부호 비트를 뺀 절대치 비트의 10진수 값을 더합니다. 가장 작은 수는 1의 보수 표현이면 $-(2^{n-1} - 1)$, 2의 보수 표현이면 -2^{n-1} 입니다.
- **방법 ②** 1의 보수(2의 보수)를 다시 1의 보수(2의 보수)로 바꾼 뒤, 그 결과를 부호화-절대치 표현으로 읽어 10진수로 나타냅니다.

방법 ①은 표 4-5의 열을 아래에서 위로 보면 이해됩니다. 4비트 1의 보수 표현에서 가장 작은 수 -7은 1000이고, 절대치 비트가 000입니다. 1001은 -6, 1010은 -5처럼 절대치 비트의 값이 1 늘 때마다 수도 1씩 커집니다. 그러니 절대치 비트의 값은 가장 작은 수에서 얼마나 떨어져 있는지를 나타내는 거리(offset)이고, 가장 작은 수에 그 거리를 더하면 값이 나옵니다. 2의 보수 표현에서는 가장 작은 수가 -8(1000)이라는 것만 다릅니다.

일반적인 n 비트로도 확인할 수 있습니다. 2의 보수 표현에서 $-m$ 의 비트열은 2진수로 읽으면 $2^n - m$ 입니다. 부호 비트의 자리값 2^{n-1} 을 빼면 절대치 비트의 값은 $2^{n-1} - m$ 이고, 여기에 -2^{n-1} 을 더하면 정확히 $-m$ 이 됩니다. 1의 보수 표현에서는 비트열이 $2^n - 1 - m$ 이므로 절대치 비트의 값이 $2^{n-1} - 1 - m$ 이고, $-(2^{n-1} - 1)$ 을 더하면 $-m$ 입니다.

방법 ②는 보수의 보수가 원래 수라는 데서 나옵니다. 비트를 두 번 뒤집으면 제자리로 돌아오니, 1장에서 $\neg\neg p \equiv p$ 였던 것과 같습니다. 2의 보수도 $2^n - (2^n - x) = x$ 이므로 두 번 구하면 원래 수가 됩니다. 부호 비트는 그대로 두고 절대치 비트만 다시 바꾸면 음수의 부호화-절대치 표현이 나오므로, 부호 비트 1을 -로, 절대치 비트를 절댓값으로 읽습니다.

예제

4bit 컴퓨터에서 1의 보수 1010과 2의 보수 1011을 10진수로 변환하시오.

풀이

1의 보수 1010 부호 비트가 1이므로 음수입니다. 방법 ①로는 4비트 1의 보수 표현의 가장 작은 수 $-(2^{4-1} - 1) = -7$ 에 절대치 비트 010의 값 $1 \times 2^1 = 2$ 를 더해 $-(2^{4-1} - 1) + (1 \times 2^1) = -5$ 입니다. 방법 ②로는 부호 비트를 두고 010을 다시 1의 보수로 바꾸어 101을 얻습니다. 1101_2 를 부호화-절대치 표현으로 읽으면 $-(1 \times 2^2 + 1 \times 2^0) = -5$ 입니다.

2의 보수 1011 방법 ①로는 가장 작은 수 $-2^{4-1} = -8$ 에 절대치 비트 011의 값 $1 \times 2^1 + 1 \times 2^0 = 3$ 을 더해 $-(2^{4-1}) + (1 \times 2^1 + 1 \times 2^0) = -5$ 입니다. 방법 ②로는 011을 다시 2의 보수로 바꿉니다. 011의 1의 보수 100에 1을 더하면 101이므로, 1101_2 를 부호화-절대치 표현으로 읽어 $-(1 \times 2^2 + 1 \times 2^0) = -5$ 입니다.

$\therefore$ 1의 보수 1010과 2의 보수 1011은 모두 -5입니다.

같은 -5가 1의 보수 표현에서는 1010, 2의 보수 표현에서는 1011로 적힌다는 것을 표 4-5에서도 확인할 수 있습니다. 방법 ②를 쓸 수 없는 비트열이 하나 있습니다. 4비트 2의 보수 1000에서 절대치 비트 000을 다시 2의 보수로 바꾸면 $111 + 1 = 1000$ 의 아래 세 자리인 000이 되어 -0처럼 읽히지만, 1000은 -8입니다. 절댓값 8은 절대치 비트 세 자리에 담을 수 없기 때문입니다. 이 비트열은 방법 ①로 $-8 + 0 = -8$ 로 읽습니다.

4. 보수의 연산

이제 4비트 컴퓨터에서 뺄셈을 보수의 덧셈으로 계산해 봅시다. 빼는 수를 음수로 바꾸어 1의 보수 표현과 2의 보수 표현으로 각각 더합니다. 아래 예제의 세 계산 모두 결과가 4비트 범위 안에 있습니다. 각 예제에서 왼쪽 계산이 1의 보수 표현, 오른쪽 계산이 2의 보수 표현입니다.

예제 ①

4bit 컴퓨터에서 $2_{10} - 4_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오.

풀이

$|2_{10}| = 0010$, $|4_{10}| = 0100$ 입니다. $2 - 4$ 를 $2 + (-4)$ 로 보고 -4를 보수 표현으로 바꿉니다. 1의 보수 표현으로 -4는 0100을 뒤집은 1011이고, 2의 보수 표현으로는 여기에 1을 더한 1100입니다.

$$\begin{array}{r} 0010 \\ + 1011 \\ \hline 1101 \end{array} \qquad \begin{array}{r} 0010 \\ + 1100 \\ \hline 1110 \end{array}$$

두 경우 모두 다섯째 자리로 넘치는 자리올림이 없습니다. 결과의 부호 비트가 1이므로 음수이고, 3-3)의 방법 ②로 읽습니다. 1의 보수 1101은 절대치 비트 101을 뒤집어 1010이 되고, 2의 보수 1110은 절대치 비트 110을 2의 보수로 바꾸어 1010이 됩니다. 부호화-절대치 표현 1010은 -2입니다.

$\therefore$ 1의 보수를 이용하면 $1101_2 \rightarrow 1010 = -2_{10}$, 2의 보수를 이용하면 $1110_2 \rightarrow 1010 = -2_{10}$

자리올림이 넘치지 않은 것은 결과가 음수이기 때문입니다. $x < y$ 이면 $x + (2^n - y) = 2^n - (y - x)$ 가 2^n 보다 작아 n 비트 밖으로 넘치지 않고, 이 값이 바로 $y - x$ 의 2의 보수, 곧 $-(y - x)$ 의 비트열입니다. 1의 보수 표현에서도 마찬가지입니다.

예제 ②

4bit 컴퓨터에서 $4_{10} - 2_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오.

풀이

$|4_{10}| = 0100$, $|2_{10}| = 0010$ 이고, -2 는 1의 보수 표현으로 1101, 2의 보수 표현으로 1110입니다.

$$\begin{array}{r} 0100 \\ + 1101 \\ \hline 10001 \\ + \quad 1 \\ \hline 0010 \end{array} \qquad \begin{array}{r} 0100 \\ + 1110 \\ \hline 10010 \\ \text{(맨 앞 1은 버림)} \end{array}$$

이번에는 두 경우 모두 다섯째 자리로 1이 넘칩니다. 4비트 컴퓨터는 넷째 자리까지만 담으므로 이 1을 처리해야 하는데, 두 표현의 처리 방법이 다릅니다. 2의 보수 표현에서는 넘친 자리올림을 버립니다. 0010이 남고 답은 2입니다. 1의 보수 표현에서는 넘친 자리올림을 버리지 않고 최하위 비트에 다시 더합니다. 남은 0001에 1을 더해 0010이 되고 답은 2입니다. 이렇게 맨 위에서 넘친 자리올림을 맨 아래로 돌려 더하는 것을 순환 자리올림(end-around carry)이라고 합니다.

∴ 두 방법 모두 $0010_2 = 2_{10}$

왜 처리가 다른지는 3-2)의 10진수 비유로 보면 분명합니다. $4 - 2$ 를 10의 보수로 계산하면 2의 10의 보수 8을 더해 $4 + 8 = 12$ 이고, 다 찬 10을 버리면 2가 남습니다. 9의 보수로 계산하면 2의 9의 보수 7을 더해 $4 + 7 = 11$ 입니다. 9의 보수는 9를 기준으로 채웠는데 넘칠 때는 10을 채워야 넘어가므로, 10을 버리면 1이 남아 하나가 모자랍니다. 그래서 버린 자리에서 1을 가져와 더해 2를 만듭니다. 2진수의 2의 보수가 10의 보수에, 1의 보수가 9의 보수에 해당합니다. 식으로 쓰면 차이가 한눈에 보입니다.

$$x - y = x + (2^n - y) - 2^n, \quad x - y = x + (2^n - 1 - y) - 2^n + 1$$

왼쪽이 2의 보수, 오른쪽이 1의 보수입니다. 2의 보수에서는 넘친 자리올림 2^n 을 버리기만 하면 되고, 1의 보수에서는 2^n 을 버린 뒤 1을 더해야 합니다. 순환 자리올림이 바로 이 +1입니다. 자리올림이 넘치지 않으면 버릴 2^n 도 보충할 1도 없으므로, 예제 ①에서는 두 표현 모두 더 할 일이 없었습니다. 3-2)에서 1의 보수 표현의 한계로 연산의 정확성을 든 것은 이 한 단계가 더 필요하다는 뜻입니다.

예제 ③

4bit 컴퓨터에서 $-2_{10} - 4_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오.

풀이

$-2 - 4 = (-2) + (-4)$ 이므로 두 음수를 더합니다. $|2_{10}| = 0010$, $|4_{10}| = 0100$ 이므로 -2 는 1의 보수 표현으로 1101, 2의 보수 표현으로 1110이고, -4 는 1의 보수 표현으로 1011, 2의 보수 표현으로 1100입니다.

$$\begin{array}{r}
 1101 \\
 + 1011 \\
 \hline
 11000 \\
 + \quad 1 \\
 \hline
 1001
 \end{array}
 \qquad
 \begin{array}{r}
 1110 \\
 + 1100 \\
 \hline
 11010 \\
 \text{(맨 앞 1은 버림)}
 \end{array}$$

두 경우 모두 자리올림이 넘칩니다. 1의 보수 표현은 넘친 1을 다시 더해 $1000 + 1 = 1001$ 이 되고, 2의 보수 표현은 넘친 1을 버려 1010이 남습니다. 결과의 부호 비트가 1이므로 3-3)의 방법 ②로 읽습니다. 1의 보수 1001은 절대치 비트 001을 뒤집어 1110이 되고, 2의 보수 1010은 절대치 비트 010을 2의 보수로 바꾸어 1110이 됩니다. 부호화-절대치 표현 1110은 -6입니다.

∴ 1의 보수를 이용하면 $1001_2 \rightarrow 1110 = -6_{10}$, 2의 보수를 이용하면 $1010_2 \rightarrow 1110 = -6_{10}$

두 결과 1001과 1010을 건주면 2의 보수 쪽이 1만큼 큼니다. 같은 수 -6의 2의 보수 표현이 1의 보수 표현에 1을 더한 것이기 때문입니다. 표 4-5의 1001과 1010 줄에서 확인할 수 있습니다.

자리올림과 초과

예제 ②와 ③에서 다섯째 자리로 넘친 1은 3-1)의 초과가 아닙니다. 넘친 1은 n 비트 밖으로 나간 자리올림일 뿐이고, 답은 범위 안에서 바르게 나왔습니다. 초과는 참값이 범위를 벗어나는 일입니다. 2의 보수 표현으로 두 경우를 보겠습니다. $5 + 4$ 는 $0101 + 0100 = 1001$ 이고, 1001은 -7입니다. 참값 9가 범위 -8부터 7을 벗어나 두 양수의 합이 음수가 되었습니다. $(-5) + (-4)$ 는 $1011 + 1100 = 10111$ 이고, 자리올림을 버린 0111은 7입니다. 참값 -9가 범위를 벗어나 두 음수의 합이 양수가 되었습니다.

$1011 + 1100$ 은 3-1)에서 부호화-절대치 표현의 $(-3) + (-4)$ 로 더했던 바로 그 비트열입니다. 부호화-절대치 표현에서는 범위 안의 -7조차 틀렸지만, 2의 보수 표현에서 이 비트열은 -5와 -4이고, 틀린 까닭은 참값 -9가 범위 밖이기 때문입니다. 2의 보수 표현에서는 3-1)에서 말한 두 모습, 곧 부호가 같은 두 수를 더했는데 결과의 부호가 바뀌는 것이 초과를 알아보는 정확한 기준이 됩니다. 자리올림이 넘쳤는지로는 초과를 판단할 수 없습니다. 예제 ②는 자리올림이 넘쳐도 초과가 아니었고, $5 + 4$ 는 자리올림이 넘치지 않아도 초과였습니다.

표 4-6은 이 장의 세 표현을 건주어 정리한 것입니다. 2의 보수 표현은 0을 하나로 나타내고 넘친 자리올림을 버리기만 하면 되므로, 오늘날 컴퓨터는 정수를 대부분 2의 보수 표현으로 저장합니다.

	부호화-절대치 표현	1의 보수 표현	2의 보수 표현
양수 $+m$	부호 비트 0, 절대치 비트에 m (셋이 같음)		
음수 $-m$	부호 비트 1, 절대치 비트에 m	$+m$ 의 비트를 모두 뒤집음	$+m$ 의 1의 보수에 1을 더함
범위	-7부터 7까지	-7부터 7까지	-8부터 7까지
0의 표현	0000, 1000	0000, 1111	0000
넘친 자리올림	비트열을 그대로 더하면 틀림	최하위 비트에 다시 더함	버림

표 4-6 세 표현의 비교(4비트)

장을 마치며

이 장의 수는 모두 한 식 위에 서 있었습니다. 각 자리의 숫자에 그 자리의 값 b^i 를 곱해 더하는 $n = \sum a_i b^i$ 입니다. 1절은 이 식으로 자연수와 실수를 적었고, 2절의 수 체계는 같은 식에서 기수 b 만 바꾼 것이었습니다. 10진수를 2진수로 바꿀 때 2로 거듭 나눈 나머지가 자리의 숫자가 된 것은 1절의 나머지 연산 그대로였고, 2진수를 세 자리나 네 자리씩 묶은 것은 $2^3 = 8$, $2^4 = 16$ 이라는 자리값의 관계 덕분이었습니다.

3절과 4절은 자리 수가 n 으로 정해질 때 무슨 일이 생기는지를 다루었습니다. n 비트 바로 위의 자리값 2^n 이 2의 보수의 기준이 되었고, 넘친 자리올림을 버리는 것이 곧 $x + (2^n - y) - 2^n = x - y$ 의 -2^n 이었습니다. 1의 보수는 기준이 $2^n - 1$ 이라 하나가 모자랐고, 그 1을 순환 자리올림으로 보충했습니다. 범위를 넘는 초과와 넘친 자리올림을 구별하는 것도 같은 그림에서 나왔습니다.

다음 장에서는 수를 하나씩 다루던 데서 나아가, 수를 가로와 세로로 늘어놓아 한꺼번에 다루는 행렬을 배웁니다. 1절에서 정리한 교환법칙, 결합법칙, 분배법칙, 항등원과 역이 행렬에서는 어떻게 되는지 견주어 보며 읽으세요. 곱에 대한 역은 5장에서 역행렬로 다시 나옵니다.

떠올려 보기

1. 589_{10} 에서 8은 a_0, a_1, a_2 가운데 무엇이고, 그 자리의 값은 얼마입니까? 한 자리의 숫자 a_i 가 $0 \leq a_i < b$ 이어야 하는 까닭은 무엇입니까?
2. 유리수를 소수로 나타내면 소수부가 끝나거나 반복되는 까닭은 무엇입니까?
3. 무리수가 덧셈과 곱셈에 닫혀 있지 않음을 보이는 예를 하나씩 들어 보세요.
4. 복소수 $3 + 6i$ 의 허수부는 얼마입니까?
5. $3 \mid 12$ 와 $3/12$ 는 어떻게 다릅니까?
6. $-7 \bmod 3$ 은 얼마입니까? -1 이 아닌 까닭은 무엇입니까?
7. 10진수를 2진수로 바꿀 때 정수부는 나머지를 아래에서 위로, 소수부는 정수부를 위에서 아래로 읽는 까닭은 무엇입니까?
8. 0.111_2 를 16진수로 바꿀 때 0을 왼쪽이 아니라 오른쪽에 채우는 까닭은 무엇입니까?
9. 4비트 2의 보수 표현의 범위가 -7 이 아니라 -8 부터인 까닭은 무엇입니까?
10. 4비트 2의 보수로 $4 - 2$ 를 계산할 때 다섯째 자리로 넘친 1은 초과입니까?
11. 1의 보수 표현의 덧셈에서 넘친 자리올림을 최하위 비트에 다시 더하는 까닭은 무엇입니까?
12. 4비트 2의 보수 1011을 3-3의 두 방법으로 10진수로 바꿔 보세요.

짧은 답

1. a_1 이고 자리값은 $10^1 = 10$ 입니다. 한 자리에 b 이상이 되면 윗자리로 올리므로, 한 자리의 숫자는 기수보다 작아야 합니다.
2. 긴 나눗셈에서 나오는 나머지가 0부터 분모보다 1 작은 수까지 유한한 종류뿐이어서, 0이 되어 끝나거나 앞에 나온 나머지가 다시 나와 같은 숫자가 되풀이되기 때문입니다.
3. 덧셈은 $\sqrt{2} + (-\sqrt{2}) = 0$, 곱셈은 $\sqrt{2} \times \sqrt{2} = 2$ 입니다. 결과가 유리수입니다.
4. 6입니다. 허수부는 $6i$ 가 아니라 i 앞의 실수입니다.
5. $3 \mid 12$ 는 '3은 12를 나눈다'는 참인 명제이고, $3/12$ 는 0.25라는 수입니다.
6. 2입니다($-7 = 3 \times (-3) + 2$). 나머지는 $0 \leq r < 3$ 이어야 하므로 -1 은 나머지가 될 수 없습니다.
7. 둘 다 처음 나온 숫자가 소수점에 가장 가까운 자리이기 때문입니다. 정수부의 첫 나머지는 a_0 이고, 소수부의 첫 정수부는 a_{-1} 입니다.

8. 소수부의 오른쪽에 붙인 0은 값을 바꾸지 않지만($0.111 = 0.1110$), 왼쪽에 붙이면 0.0111 이라는 다른 수가 되기 때문입니다. 0.1110 이 $0.E$ 입니다.
9. 0을 0000 하나로만 나타내므로 비트열 하나가 남고, 그 남은 1000이 $-7(1001)$ 보다 1 작은 -8 을 나타내기 때문입니다.
10. 초과가 아닙니다. 4비트 밖으로 넘친 자리올림일 뿐이고, 버리면 답 $0010 = 2$ 가 범위 안에서 바르게 나옵니다.
11. 1의 보수는 $2^n - 1$ 을 채우는 수인데, 넘칠 때는 2^n 을 채워야 넘어가므로 넘친 자리올림을 버리면 1이 모자라기 때문입니다. 10진수의 9의 보수와 같습니다.
12. 방법 ①은 $-2^3 + 3 = -5$, 방법 ②는 부호 비트를 두고 011을 2의 보수 101로 바꾼 1101을 부호화-절대치 표현으로 읽어 -5 입니다.